# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
## FACULTAD DE INGENIERÍA

**Bases de datos. Grupo 1**

**Profesor:Fernando Arreola Franco**
*fernando.arreola@fi.unam.edu*

**Fecha de entrega:** Viernes 30 de octubre del 2026

**Tarea 11**

# TIPOS DE DATOS POSTGREST

**Alumna:** Yadira Colima Esquivel

**Numero de cuenta:** 422115533

**Semestre** 2027-1

# Tipos de Datos en PostgreSQL

## 1. Tipos Numericos

Los tipos numericos en PostgreSQL se dividen en enteros de diferentes capacidades y tipos de precision exacta o flotante [1].

- smallint (Entero pequeño): Ocupa 2 bytes de almacenamiento. Su rango de valores validos va desde -32,768 hasta 32,767 [1]. Representa numeros enteros sin decimales [1]. Ejemplo de uso: 25000 [1].
- integer o int (Entero estandar): Ocupa 4 bytes. Su rango abarca desde -2,147,483,648 hasta 2,147,483,647 [1]. Es el tipo de dato entero mas utilizado para identificadores y contadores estandar. Ejemplo de uso: 1048576 [1].
- bigint (Entero grande): Ocupa 8 bytes. Permite almacenar un rango mas amplio desde -9,223,372,036,854,775,80 hasta 9,223,372,036,854,775,807 [1]. Ideal para tablas con millones de registros. Ejemplo: 9876543210.
- numeric o decimal (Precision exacta): Permite almacenar numeros con una precision arbitraria de hasta 131,000 digitos antes del punto decimal y hasta 16,383 despues [1]. Es el tipo recomendado para aplicaciones financieras y monetarias donde los errores de redondeo de punto flotante no son aceptables [1]. Ejemplo de uso: 199.99 [1].
- real y double precision (Punto flotante inexacto): Ocupan 4 y 8 bytes respectivamente. Almacenan numeros con precision inexacta basados en binarios de punto flotante. Ejemplo: 3.14159.

## 2. Tipos de Caracter (Texto)

PostgreSQL ofrece tres tipos principales para almacenar cadenas de caracteres de texto [1].

- character varying(n) o varchar(n) (Caracteres de longitud variable): Almacena cadenas de texto de hasta n caracteres [1]. Si se excede la longitud declarada, el sistema arroja un error a menos que se trunque. Ejemplo: 'Ingenieria en Sistemas' [1].
- character(n) o char(n) (Caracteres de longitud fija): Almacena cadenas de longitud fija rellenando con espacios en blanco a la derecha si el texto ingresado es mas corto que n [1]. Ejemplo: 'ACTIVO'.
- text (Texto de longitud ilimitada): Permite almacenar cadenas de texto sin un limite practico de longitud (hasta 1 GB por campo de manera individual) [1]. Es altamente eficiente para almacenar descripciones, articulos o codigos largos. Ejemplo: 'Este campo permite almacenar descripciones extensas, reportes tecnicos o registros de bitacora detallados' [1].

## 3. Tipos de Fecha y Hora

Permiten la gestion cronologica precisa de eventos y registros temporales [1].

- date (Fecha): Ocupa 4 bytes. Almacena unicamente fechas civiles (ano, mes y dia) abarcando desde el ano 4713 a.C. hasta el 587489 d.C. bajo el formato estandar YYYY-MM-DD [1]. Ejemplo de uso: '2026-11-03' [1].
- time (Hora del dia): Ocupa 8 bytes. Almacena la hora del dia sin fecha asociada con una resolucion de hasta microsegundos en formato HH:MM:SS. Ejemplo: '21:40:00'.

- timestamp (Fecha y hora sin zona horaria): Ocupa 8 bytes. Almacena tanto la fecha como la hora combinadas con una precision de microsegundos, omitiendo cualquier referencia a zonas horarias geograficas [1]. Formato YYYY-MM-DD HH:MM:SS [1]. Ejemplo: '2026-11-03 21:40:00' [1].

- timestamptz (Fecha y hora con zona horaria): Ocupa 8 bytes. Convierte de forma automatica los valores ingresados a la hora universal coordinada (UTC) para su almacenamiento interno y los ajusta segun la zona horaria de la sesion del cliente.

## 4. Otros 3 Tipos de Datos de Alto Interes

- jsonb (JSON Binario): Almacena datos en formato JSON de manera descompuesta dentro de una estructura binaria interna. Aunque requiere un mayor costo de procesamiento inicial al insertar, permite realizar consultas avanzadas, busquedas de claves y operaciones de indexacion con indices GIN de forma ultrarrapida, soportando hasta 1 GB por valor. Ejemplo: '"sensor": "DHT22", .ªctivo": true, "pines": [1, 4]'::jsonb.

- uuid (Identificador Unico Universal): Ocupa 16 bytes. Almacena identificadores unicos a nivel global estandarizados bajo el formato RFC 4122. Evita colisiones de datos en sistemas distribuidos y reemplaza eficazmente a las secuencias numericas tradicionales [1]. Ejemplo: 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11' [1].

- inet (Direccion de Red IP): Ocupa de 7 a 19 bytes. Almacena direcciones de red IPv4 o IPv6 junto con su respectiva mascara de subred de manera nativa, facilitando operaciones de filtrado y validacion de subredes en consultas SQL [1]. Ejemplo: '192.168.1.50/24' [1].

# Ejemplos de Uso en Creacion de Bases de Datos y Tablas

A continuacion se presenta la implementacion practica en codigo SQL para la creacion de una base de datos corporativa y una tabla relacional que integra los tipos de datos estudiados:

Listing 1: Creacion de Base de Datos y Tabla en PostgreSQL

```sql
-- Creacion de la base de datos principal
CREATE DATABASE sistema_ingenieria
    WITH
    OWNER = postgres
    ENCODING = 'UTF8'
    CONNECTION LIMIT = -1;

-- Conectarse a la base de datos e implementar la tabla relacional
CREATE TABLE dispositivos_monitoreo (
    id_dispositivo uuid PRIMARY KEY,
    codigo_interno smallint NOT NULL,
    nombre_equipo varchar(100) NOT NULL,
    especificaciones text,
    costo_adquisicion numeric(10, 2),
    ip_conexion inet,
    configuracion_json jsonb,
    fecha_instalacion date,
    ultima_actualizacion timestamp
);
```

Listing 2: Ejemplo de Insercion de Registros

```sql
INSERT INTO dispositivos_monitoreo (
    id_dispositivo,
    codigo_interno,
    nombre_equipo,
    especificaciones,
    costo_adquisicion,
    ip_conexion,
    configuracion_json,
```

```
    fecha_instalacion,
    ultima_actualizacion
) VALUES (
    'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11',
    101,
    'Sensor Ambiental Planta Baja',
    'Sensor IoT dedicado para la lectura de temperatura y humedad relativa.',
    1250.50,
    '192.168.1.50/24',
    '{"modelo": "DHT22", "umbral_alerta": 35.5, "pines": [1, 4]}',
    '2026-11-03',
    '2026-11-03 21:40:00'
);
```

# Referencias


[1] The PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation: Chapter 8. Data Types*. Recuperado de `https://www.postgresql.org/docs/current/datatype.html` [1]

[2] Douglas, K., & Douglas, S. (2003). *Beginning PostgreSQL*. Apress.